

Hypothesis 7: Does Sentence Type Interact with Voice on Recognition Accuracy?

2 x 4 Non-Parametric Factorial Analysis: Voice (Active/Passive) x Sentence Type (HH/HL/LH/LL)

Team: Metamatics

May 06, 2026

Contents

1	What is Hypothesis 7?	2
1.1	The Big Idea	2
1.2	The Question	3
2	Formal Hypothesis	3
3	Step A: Loading the Data	3
3.1	Extracting the Sentence Type (HH / HL / LH / LL)	5
4	Step B: Descriptive Statistics	6
4.1	Table 1 - Mean Accuracy per Cell	6
4.2	Table 2 - Summary by Sentence Type	7
5	Step C: Visualising the Data	8
5.1	Figure 1 - Bar Chart	8
5.2	Figure 2 - Interaction Plot	8
5.3	Figure 3 - Violin Plots	11
5.4	Figure 4 - Heatmap	12
5.5	Figure 5 - Faceted Boxplots	12
6	Step D: Normality Check	14
6.1	Q-Q Plots for Each Cell	15
6.2	Test Selection Decision	15
6.2.1	Which test will be used?	15

7 Step E: Main Statistical Test	15
8 Step F: Post-Hoc Tests	18
9 Step G: Complete From-Scratch Analysis — Voice $\times$ Lexical Interaction on Corrected Memorability	19
9.1 Data Preparation (Cell Isolation)	19
9.2 Reference-Style Exploratory Plots	21
9.2.1 Plot R1 — Corrected Memorability Score by Sentence Type and Voice	21
9.2.2 Plot R2 — Distribution of Corrected Memorability Scores (Violin)	22
9.2.3 Plot R3 — Recognition Hit Rate by Sentence Type and Voice	23
9.2.4 Plot R4 — Reaction Time for Correctly Detected Targets (Boxplot)	25
9.3 Step 1 — Per-Group Normality Testing (8 Independent Shapiro-Wilk Tests)	27
9.3.1 Q-Q Plot Grid (2×4 — one panel per cell)	28
9.3.2 Density Plot	30
9.4 Step 2 — Statistical Decision Logic	30
9.4.1 Test Chosen: Scheirer-Ray-Hare Test (Non-Parametric 2-way ANOVA) .	32
9.5 Step 3 — High-Accuracy Interaction Plot	32
9.6 Step 4 — The “Passive Penalty” Bar Chart	33
9.7 Step 5 — Plain English Conclusion	34
9.7.1 Does the difficulty of Passive voice significantly increase when the words are Low-Memorability (LL)?	34

1 What is Hypothesis 7?

1.1 The Big Idea

In our experiment, every sentence was built using two nouns:

- **The Subject noun** (the “doer” in the sentence)
- **The Object noun** (the “receiver” in the sentence)

Each noun was either **High** (H) or **Low** (L) memorability. This gives us **4 sentence types**:

Code	Subject	Object	Meaning
HH	High	High	Easy to remember both nouns
HL	High	Low	Easy subject, hard object
LH	Low	High	Hard subject, easy object
LL	Low	Low	Difficult to remember both

We also have **2 voice conditions**: Active and Passive. Full design = **2 (Voice) × 4 (Sentence Type) = 8 combinations**.

1.2 The Question

Does the difficulty of remembering sentence wording depend **BOTH** on voice **AND** on noun memorability? If passive voice hurts memory differently depending on word difficulty, there is an **interaction effect**.

2 Formal Hypothesis

H0: Voice effect on accuracy is the SAME for all sentence types. No interaction.

H7: Voice effect on accuracy DIFFERS depending on sentence type. Interaction exists.

3 Step A: Loading the Data

```
analysis_path <- file.path(output_dir, "H3H4_step3_analysis.csv")
if (!file.exists(analysis_path)) {
  stop("H3H4_step3_analysis.csv not found in R_output/. Please knit Hypo3and4_analysis.Rmd first")
}

df_raw <- read_csv(analysis_path, show_col_types = FALSE)

cat(sprintf("Loaded: %s rows, %d participants\n",
            format(nrow(df_raw), big.mark = ","),
            n_distinct(df_raw$participant_ID)))
```

```
## Loaded: 4,478 rows, 112 participants
```

```
cat("\n=== ALL COLUMN NAMES ===\n")
```

```
##
## === ALL COLUMN NAMES ===
```

```
cat(paste("-", names(df_raw), collapse = "\n"))
```

```
## - participant_ID
## - Core_ID
## - Test_Voice
## - WR_Accuracy
## - WR_RT
## - Original_Voice
## - Condition
## - Voice_Label
## - corrected_memorability
```

```
cat("\n\n=== FIRST 5 ROWS ===\n")
```

```
##
##
## === FIRST 5 ROWS ===
```

```
print(head(df_raw, 5))
```

```
## # A tibble: 5 x 9
##   participant_ID Core_ID Test_Voice WR_Accuracy WR_RT Original_Voice Condition
##           <dbl> <chr>   <chr>         <dbl> <dbl> <chr>         <chr>
## 1           232 HH_137 P             1  1535 P             Identical
## 2           232 HH_138 A             1   401 A             Identical
## 3           232 HH_139 P             1   655 P             Identical
## 4           232 HH_140 P             1   990 A             Transformed
## 5           232 HH_142 A             1  1841 A             Identical
## # i 2 more variables: Voice_Label <chr>, corrected_memorability <dbl>
```

```
cat("\n=== UNIQUE Core_ID values (first 20) ===\n")
```

```
##
## === UNIQUE Core_ID values (first 20) ===
```

```
print(head(unique(df_raw$Core_ID), 20))
```

```
## [1] "HH_137" "HH_138" "HH_139" "HH_140" "HH_142" "HH_144" "HH_145"
## [8] "HH_149" "HH_150" "HH_155" "HH_172" "HH_179" "HVL_102" "HVL_103"
## [15] "HVL_106" "HVL_107" "HVL_111" "HVL_113" "HVL_114" "HVL_116"
```

```

if ("Original_Voice" %in% names(df_raw)) {
  cat("\n=== UNIQUE Original_Voice values ===\n")
  print(unique(df_raw$Original_Voice))
}

```

```

##
## === UNIQUE Original_Voice values ===
## [1] "P" "A"

```

```

type_cols <- names(df_raw)[sapply(names(df_raw), function(col) {
  vals <- unique(na.omit(as.character(df_raw[[col]])))
  any(vals %in% c("HH", "HL", "LH", "LL"))
}]]
if (length(type_cols) > 0) {
  cat(sprintf("\n=== Columns with HH/HL/LH/LL values: %s ===\n", paste(type_cols, collapse=" ")))
} else {
  cat("\n=== NO column found with HH/HL/LH/LL - will extract from Core_ID ===\n")
}

```

```

##
## === NO column found with HH/HL/LH/LL - will extract from Core_ID ===

```

3.1 Extracting the Sentence Type (HH / HL / LH / LL)

```

type_col <- names(df_raw)[sapply(names(df_raw), function(col) {
  vals <- unique(na.omit(as.character(df_raw[[col]])))
  any(vals %in% c("HH", "HL", "LH", "LL"))
}]]

if (length(type_col) > 0) {
  sentence_type_vec <- as.character(df_raw[[type_col[1]]])
} else {
  raw_type <- str_extract(df_raw$Core_ID, "(?i)(HH|HL|LH|LL|HVL|LVH|LVL)")
  sentence_type_vec <- case_when(
    toupper(raw_type) == "HVL" ~ "HL",
    toupper(raw_type) == "LVH" ~ "LH",
    toupper(raw_type) == "LVL" ~ "LL",
    toupper(raw_type) %in% c("HH", "HL", "LH", "LL") ~ toupper(raw_type),
    TRUE ~ NA_character_
  )
}

cat(sprintf("Sentence types found: %s\nNA count: %d / %d\n",
  paste(head(na.omit(unique(sentence_type_vec)), 20), collapse=" "),
  sum(is.na(sentence_type_vec)), length(sentence_type_vec)))

```

```
## Sentence types found: HH, HL, LH, LL
## NA count: 0 / 4478
```

```
df <- df_raw %>%
  mutate(
    Sentence_Type = sentence_type_vec,
    Voice_Label = case_when(
      tolower(trimws(Original_Voice)) == "a" ~ "Active",
      tolower(trimws(Original_Voice)) == "p" ~ "Passive",
      TRUE ~ NA_character_
    )
  ) %>%
  filter(!is.na(Sentence_Type), !is.na(WR_Accuracy), !is.na(Voice_Label)) %>%
  mutate(Sentence_Type = factor(Sentence_Type), Voice_Label = factor(Voice_Label)) %>%
  droplevels()

cat(sprintf("\nRows after filtering: %s\nVoice levels: %s\nSentence Type levels: %s\n",
  format(nrow(df), big.mark=","),
  paste(levels(df$Voice_Label), collapse=","),
  paste(levels(df$Sentence_Type), collapse=",")))
```

```
##
## Rows after filtering: 4,478
## Voice levels: Active, Passive
## Sentence Type levels: HH, HL, LH, LL
```

```
if (nrow(df) > 0) { cat("\nCell counts:\n"); print(table(df$Sentence_Type, df$Voice_Label)) }
```

```
##
## Cell counts:
##
##      Active Passive
## HH      571      572
## HL      561      584
## LH      556      567
## LL      529      538
```

4 Step B: Descriptive Statistics

4.1 Table 1 - Mean Accuracy per Cell

```

if (nrow(df) == 0) {
  cat("No data available.\n")
  desc <- tibble(Voice_Label=character(), Sentence_Type=character(),
                 N=integer(), Mean_Acc=numeric(), Median_Acc=numeric(), SD=numeric(), SE=numeric())
} else {
  desc <- df %>%
    group_by(Voice_Label, Sentence_Type) %>%
    summarise(N=n(), Mean_Acc=round(mean(WR_Accuracy, na.rm=TRUE),4),
              Median_Acc=round(median(WR_Accuracy, na.rm=TRUE),4),
              SD=round(sd(WR_Accuracy, na.rm=TRUE),4),
              SE=round(sd(WR_Accuracy, na.rm=TRUE)/sqrt(n()),4), .groups="drop")

  desc %>%
    kable(caption="Table 1. Descriptive Statistics per Voice x Sentence Type",
          col.names=c("Voice", "Type", "N", "Mean Acc", "Median Acc", "SD", "SE"), booktabs=TRUE) %>%
    kable_styling(latex_options=c("striped", "HOLD_position"),
                  bootstrap_options=c("striped", "hover"), full_width=FALSE, position="center")
  row_spec(0, bold=TRUE, background=HDR_COLOR, color="white")
}

```

Table 2: Table 1. Descriptive Statistics per Voice x Sentence Type

Voice	Type	N	Mean Acc	Median Acc	SD	SE
Active	HH	571	0.7758	1	0.4174	0.0175
Active	HL	561	0.7718	1	0.4200	0.0177
Active	LH	556	0.8076	1	0.3946	0.0167
Active	LL	529	0.7637	1	0.4252	0.0185
Passive	HH	572	0.7395	1	0.4393	0.0184
Passive	HL	584	0.7329	1	0.4428	0.0183
Passive	LH	567	0.7372	1	0.4405	0.0185
Passive	LL	538	0.7045	1	0.4567	0.0197

4.2 Table 2 - Summary by Sentence Type

```

if (nrow(df) > 0) {
  df %>%
    group_by(Sentence_Type) %>%
    summarise(N=n(), Mean_Acc=round(mean(WR_Accuracy, na.rm=TRUE),4),
              SD=round(sd(WR_Accuracy, na.rm=TRUE),4), .groups="drop") %>%
    kable(caption="Table 2. Accuracy by Sentence Type (collapsed across Voice)",
          col.names=c("Sentence Type", "N", "Mean Accuracy", "SD"), booktabs=TRUE) %>%
    kable_styling(latex_options=c("striped", "HOLD_position"),

```

```

        bootstrap_options=c("striped","hover"), full_width=FALSE, position="center")
    row_spec(0, bold=TRUE, background=HDR_COLOR, color="white")
}

```

Table 3: Table 2. Accuracy by Sentence Type (collapsed across Voice)

Sentence Type	N	Mean Accuracy	SD
HH	1143	0.7577	0.4287
HL	1145	0.7520	0.4321
LH	1123	0.7720	0.4197
LL	1067	0.7338	0.4422

5 Step C: Visualising the Data

5.1 Figure 1 - Bar Chart

```

if (nrow(desc) == 0) { cat("Figure 1 skipped.\n") } else {
  # Join individual data for jittering over the summary bars
  p1 <- ggplot() +
    geom_col(data=desc, aes(x=Sentence_Type, y=Mean_Acc, fill=Voice_Label, group=Voice_Label),
             position=position_dodge(0.75), width=0.65, alpha=0.85, colour="black", linewidth=0.8) +
    geom_jitter(data=df, aes(x=Sentence_Type, y=WR_Accuracy, fill=Voice_Label, group=Voice_Label),
                position=position_jitterdodge(jitter.width=0.15, dodge.width=0.75),
                alpha=0.25, size=1.5, shape=21, colour="grey30", show.legend=FALSE) +
    geom_errorbar(data=desc, aes(x=Sentence_Type, ymin=Mean_Acc-SE, ymax=Mean_Acc+SE, group=Voice_Label),
                  position=position_dodge(0.75), width=0.25, colour="black", linewidth=0.8) +
    scale_fill_manual(values=PAL_VOICE, name="Voice") +
    scale_y_continuous(limits=c(0,1), labels=percent_format()) +
    labs(title="Figure 1: WR Accuracy by Voice and Sentence Type",
          subtitle="Bars = group mean | Error bars = \u00b11 SE | Dots = individual participants",
          x="Sentence Type", y="Mean WR Accuracy")
  print(p1)
  ggsave(file.path(output_dir,"H7_bar_8cells.png"), p1, width=7, height=5, dpi=150)
}

```

5.2 Figure 2 - Interaction Plot

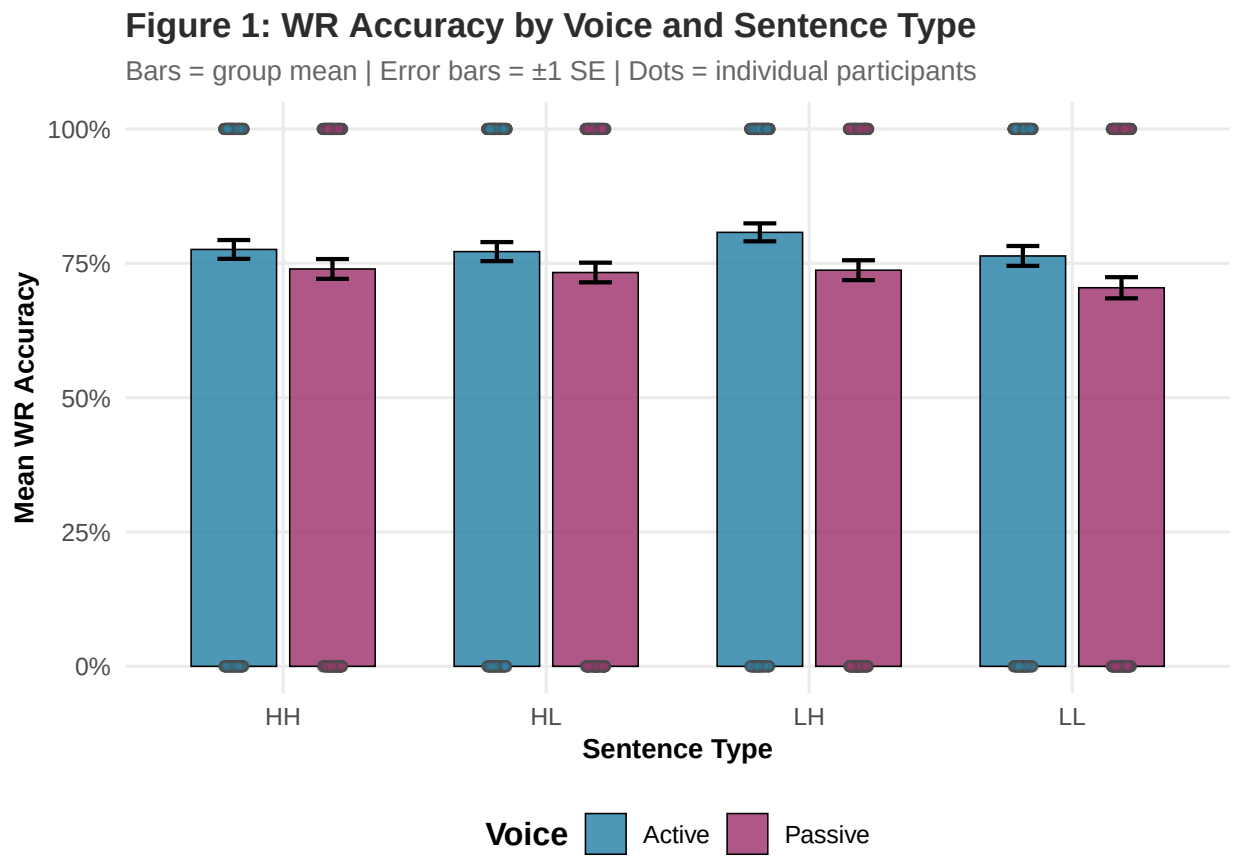


Figure 1: Figure 1. Mean WR accuracy per Voice x Sentence Type cell.

```

if (nrow(desc) == 0) { cat("Figure 2 skipped.\n") } else {
  p2 <- ggplot(desc, aes(x=Sentence_Type, y=Mean_Acc, colour=Voice_Label, group=Voice_Label)) +
    geom_line(linewidth=1.5, alpha=0.8) +
    geom_errorbar(aes(ymin=Mean_Acc-SE, ymax=Mean_Acc+SE), width=0.15, linewidth=1, alpha=0.7) +
    geom_point(size=5, shape=21, aes(fill=Voice_Label), colour="white", stroke=1.5) +
    scale_colour_manual(values=PAL_VOICE, name="Voice") +
    scale_fill_manual(values=PAL_VOICE, name="Voice") +
    scale_y_continuous(limits=c(0,1), labels=percent_format()) +
    labs(title="Figure 2: Interaction Plot -- Voice \u00d7 Sentence Type",
         subtitle="Parallel lines = NO interaction. Crossing = INTERACTION exists.",
         x="Sentence Type", y="Mean WR Accuracy")
  print(p2)
  ggsave(file.path(output_dir, "H7_interaction_plot.png"), p2, width=7, height=5, dpi=150)
}

```

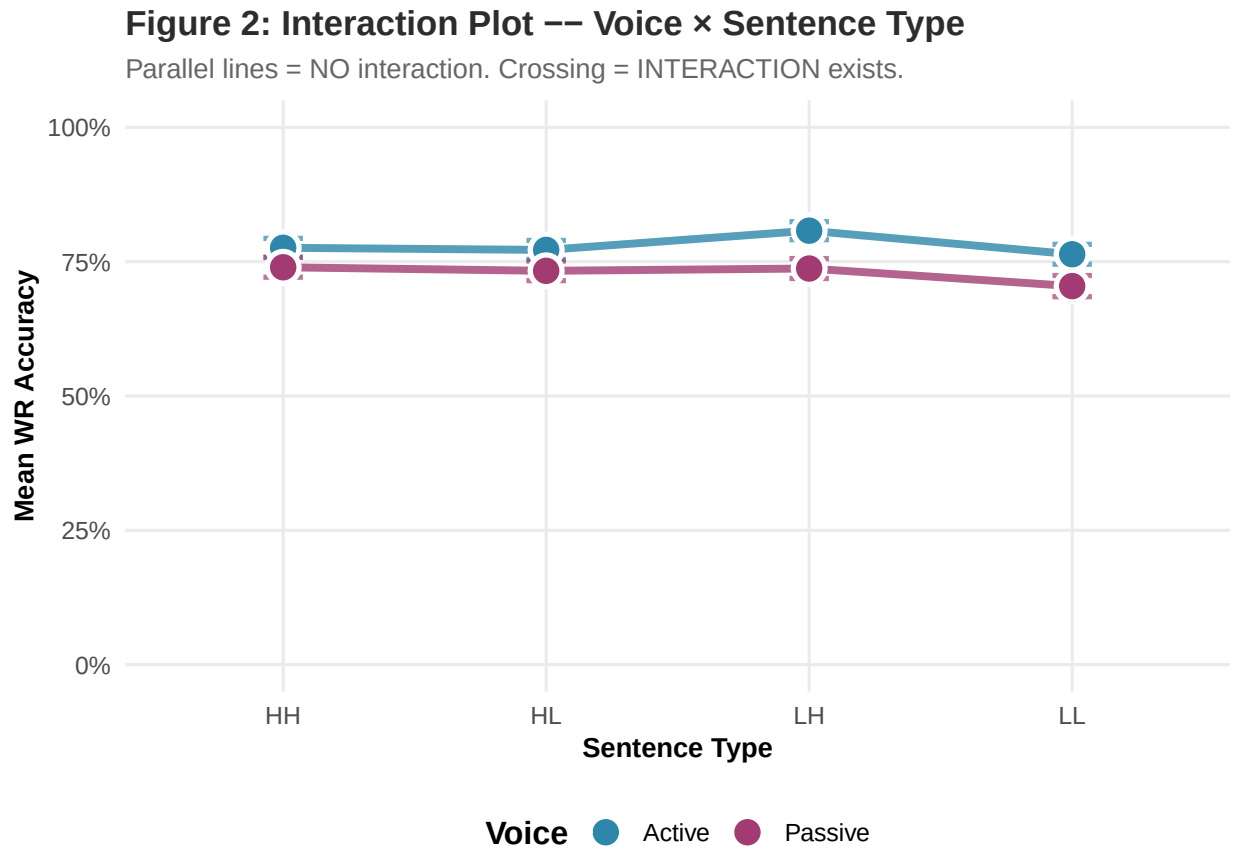


Figure 2: Figure 2. Interaction plot.

5.3 Figure 3 - Violin Plots

```
if (nrow(df) == 0) { cat("Figure 3 skipped.\n") } else {
  p3 <- ggplot(df, aes(x=Sentence_Type, y=WR_Accuracy, fill=Voice_Label)) +
    geom_violin(alpha=0.35, position=position_dodge(0.8), width=0.75, colour="black", linewidth=1) +
    geom_boxplot(width=0.18, alpha=0.85, position=position_dodge(0.8), outlier.shape=NA, colour="black") +
    geom_jitter(aes(colour=Voice_Label), position=position_jitterdodge(jitter.width=0.1, dodge.width=0.8),
                alpha=0.4, size=1.8, show.legend=FALSE) +
    stat_summary(fun=mean, geom="point", position=position_dodge(0.8), shape=23, size=4, fill="black") +
    scale_fill_manual(values=PAL_VOICE, name="Voice") +
    scale_colour_manual(values=PAL_VOICE, name="Voice") +
    scale_y_continuous(limits=c(0,1.05), labels=percent_format(), breaks=seq(0,1,0.25)) +
    labs(title="Figure 3: Accuracy Distribution by Sentence Type and Voice",
         subtitle="Violin = full distribution | Box = median & IQR | Diamond = mean | Dots = participants",
         x="Sentence Type", y="WR Accuracy")
  print(p3)
  ggsave(file.path(output_dir, "H7_violin.png"), p3, width=7, height=5, dpi=150)
}
```

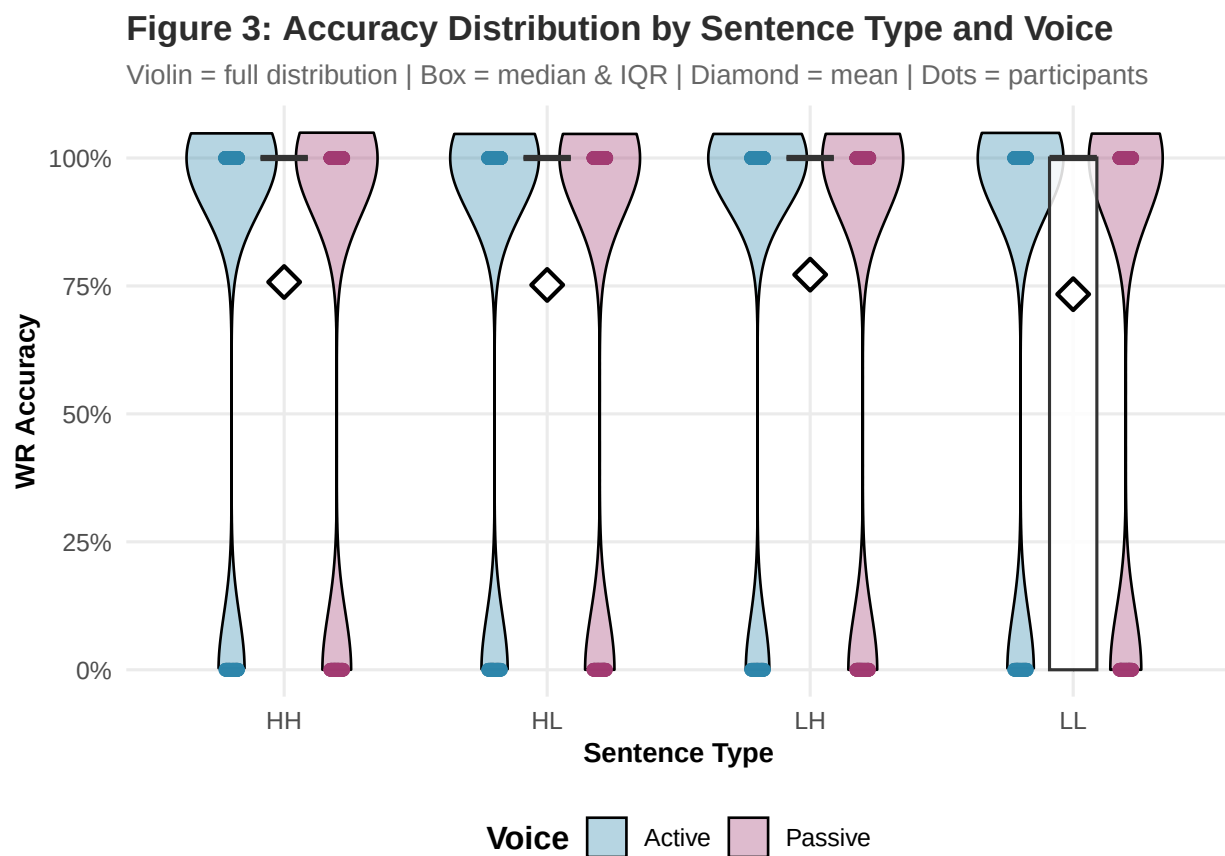


Figure 3: Figure 3. Accuracy distribution.

5.4 Figure 4 - Heatmap

```
if (nrow(desc) == 0) { cat("Figure 4 skipped.\n") } else {
  p4 <- ggplot(desc, aes(x=Sentence_Type, y=Voice_Label, fill=Mean_Acc)) +
    geom_tile(colour="white", linewidth=1.5, width=0.9, height=0.9) +
    geom_text(aes(label=scales::percent(Mean_Acc, accuracy=0.1)),
              size=5, fontface="bold", colour="white") +
    scale_fill_gradient(low="#2E4057", high="#06D6A0", name="Mean\nAccuracy", labels=percent_f
    labs(title="Figure 4: Heatmap of Mean WR Accuracy", x="Sentence Type", y="Voice")
  print(p4)
  ggsave(file.path(output_dir, "H7_heatmap.png"), p4, width=7, height=4, dpi=150)
}
```

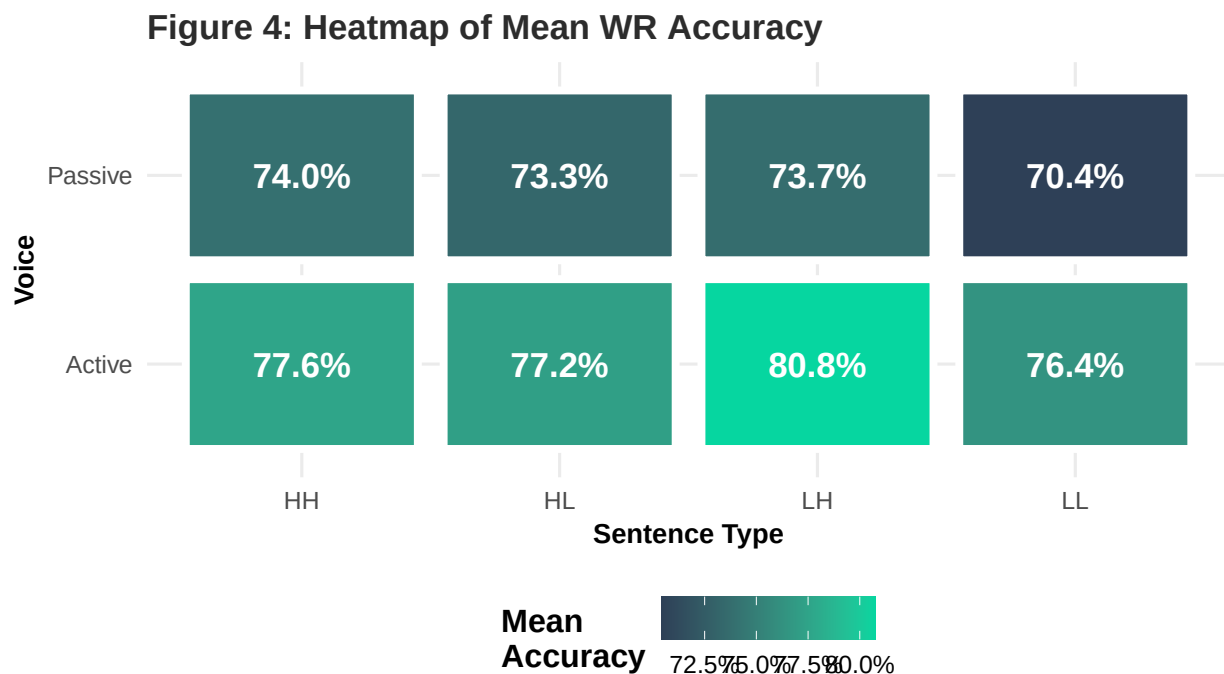


Figure 4: Figure 4. Heatmap of mean accuracy.

5.5 Figure 5 - Faceted Boxplots

```
if (nrow(df) == 0 || nlevels(df$Sentence_Type) < 1) { cat("Figure 5 skipped.\n") } else {
  df_plot5 <- df %>% mutate(Sentence_Type=droplevels(Sentence_Type))
  type_labels <- setNames(
    paste0(levels(df_plot5$Sentence_Type), "\n",
           c(HH="(both easy)", HL="(easy-hard)", LH="(hard-easy)", LL="(both hard)")[levels(df_pl
    levels(df_plot5$Sentence_Type))
}
```

```

p5 <- ggplot(df_plot5, aes(x=Voice_Label, y=WR_Accuracy, fill=Voice_Label)) +
  geom_violin(alpha=0.25, width=0.9, trim=FALSE, colour="grey40") +
  geom_boxplot(width=0.25, alpha=0.85, outlier.shape=NA, colour="grey25") +
  geom_jitter(aes(colour=Voice_Label), width=0.1, alpha=0.35, size=1.5, show.legend=FALSE) +
  stat_summary(fun=mean, geom="point", shape=23, size=4, fill="white", colour="black", stroke="black") +
  scale_fill_manual(values=PAL_VOICE, guide="none") +
  scale_colour_manual(values=PAL_VOICE, guide="none") +
  scale_y_continuous(limits=c(0,1.05), labels=percent_format()) +
  facet_wrap(~Sentence_Type, nrow=1, labeller=labeler(Sentence_Type=type_labels)) +
  labs(title="Figure 5: Active vs Passive - faceted by Sentence Type",
       subtitle="Diamond = group mean | Box = median & IQR | Dots = individuals",
       x="Voice", y="WR Accuracy")
print(p5)
ggsave(file.path(output_dir,"H7_facet_boxplots.png"), p5, width=7, height=5, dpi=150)
}

```

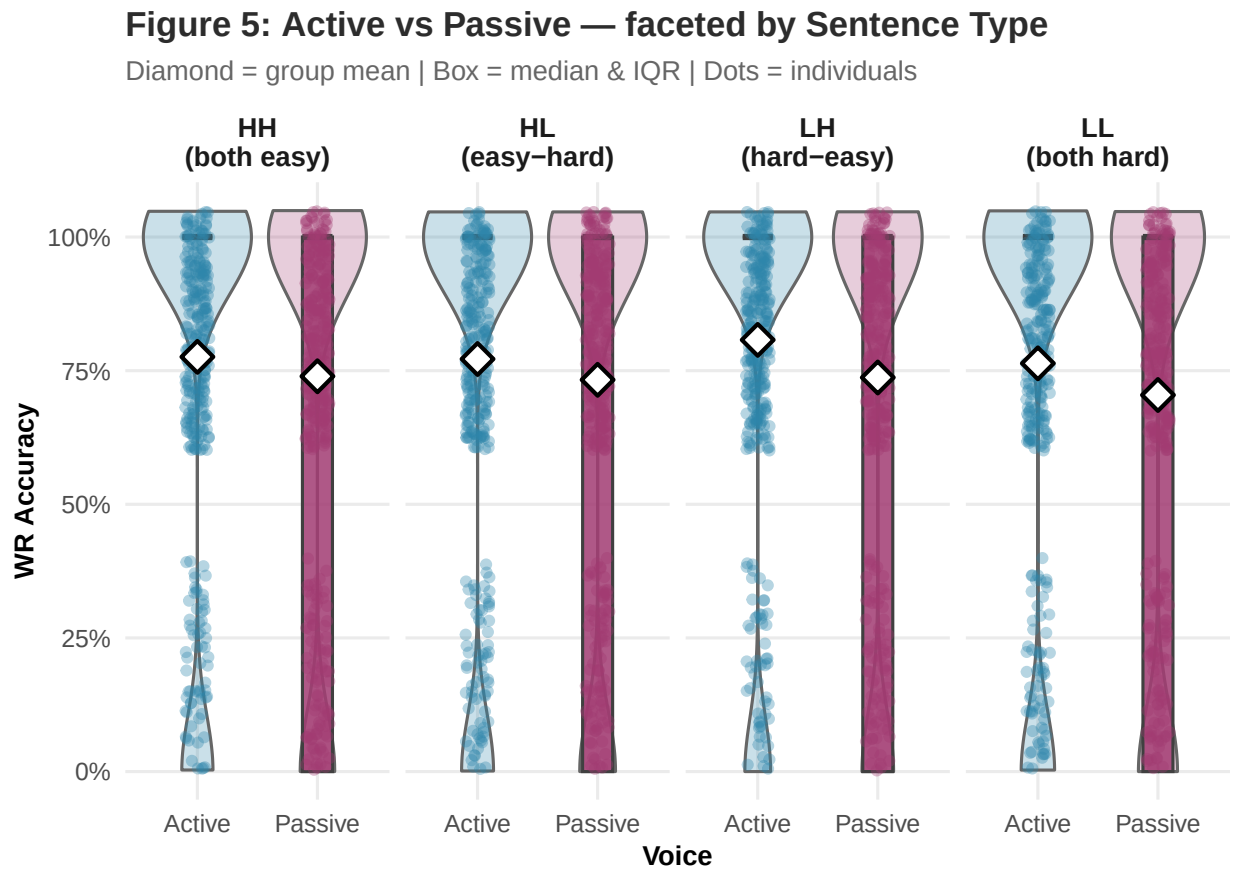


Figure 5: Figure 5. One panel per sentence type.

6 Step D: Normality Check

```
df_person <- df %>%
  group_by(participant_ID, Voice_Label, Sentence_Type) %>%
  summarise(mean_acc=mean(WR_Accuracy, na.rm=TRUE), .groups="drop")

cells <- df_person %>% distinct(Voice_Label, Sentence_Type)

sw_results <- df_person %>%
  group_by(Voice_Label, Sentence_Type) %>%
  summarise(
    n_obs = n(),
    W = if(n() >= 3) round(shapiro.test(mean_acc)$statistic, 4) else NA_real_,
    p_sw = if(n() >= 3) round(shapiro.test(mean_acc)$p.value, 4) else NA_real_,
    .groups = "drop"
  ) %>%
  mutate(Normal = case_when(
    is.na(p_sw) ~ "Too few obs",
    p_sw > 0.05 ~ "Yes (p > 0.05)",
    TRUE ~ "No (p <= 0.05)"
  )) %>%
  mutate(
    cell_col = case_when(Normal=="Yes (p > 0.05)" ~ "#1B5E20", Normal=="No (p <= 0.05)" ~ "#B71C1C",
    cell_bg = case_when(Normal=="Yes (p > 0.05)" ~ "#E8F5E9", Normal=="No (p <= 0.05)" ~ "#F8BBD0"
  )

sw_results %>%
  select(Voice_Label, Sentence_Type, n_obs, W, p_sw, Normal) %>%
  kable(caption="Table 3. Shapiro-Wilk Normality Test - 8 cells",
        col.names=c("Voice", "Type", "N obs", "W", "p-value", "Normal?"), booktabs=TRUE) %>%
  kable_styling(latex_options=c("striped", "HOLD_position"),
                bootstrap_options=c("striped", "hover"), full_width=FALSE, position="center") %>%
  column_spec(6, bold=TRUE, color=sw_results$cell_col, background=sw_results$cell_bg) %>%
  row_spec(0, bold=TRUE, background=HDR_COLOR, color="white")
```

Table 4: Table 3. Shapiro-Wilk Normality Test — 8 cells

Voice	Type	N obs	W	p-value	Normal?
Active	HH	112	0.8997	0	No (p <= 0.05)
Active	HL	112	0.8719	0	No (p <= 0.05)
Active	LH	111	0.8748	0	No (p <= 0.05)
Active	LL	111	0.9072	0	No (p <= 0.05)
Passive	HH	112	0.9253	0	No (p <= 0.05)
Passive	HL	112	0.9205	0	No (p <= 0.05)
Passive	LH	112	0.9174	0	No (p <= 0.05)

Passive	LL	112	0.9170	0	No (p <= 0.05)
---------	----	-----	--------	---	----------------

```
any_non_normal <- any(sw_results$Normal=="No (p <= 0.05)", na.rm=TRUE)
cat(sprintf("\nAny cell non-normal? %s\n",
           ifelse(any_non_normal,"YES -- will use Scheirer-Ray-Hare","NO -- may use 2-way ANOVA")))

##
## Any cell non-normal? YES -- will use Scheirer-Ray-Hare
```

6.1 Q-Q Plots for Each Cell

```
n_cells <- nrow(cells)
qq_list <- vector("list", n_cells)
for (i in seq_len(n_cells)) {
  v <- as.character(cells$Voice_Label[i])
  st <- as.character(cells$Sentence_Type[i])
  d <- df_person %>% filter(Voice_Label==v, Sentence_Type==st)
  lbl_row <- sw_results %>% filter(as.character(Voice_Label)==v, as.character(Sentence_Type)==st)
  lbl <- if (nrow(lbl_row)>0) lbl_row$Normal[1] else "No result"
  qq_list[[i]] <- ggplot(d, aes(sample=mean_acc)) +
    stat_qq(colour=PAL_VOICE[v], alpha=0.7, size=2) +
    stat_qq_line(colour="#C73E1D", linewidth=1) +
    labs(title=paste(v,"-",st), subtitle=lbl, x="Expected", y="Actual Acc")
}
qq_grid <- wrap_plots(qq_list, ncol=min(4,n_cells))
print(qq_grid)

ggsave(file.path(output_dir,"H7_qq_grid.png"), qq_grid,
        width=7, height=ceiling(n_cells/4)*3.5, dpi=150)
```

6.2 Test Selection Decision

6.2.1 Which test will be used?

Scheirer-Ray-Hare Test — at least one cell failed normality.

7 Step E: Main Statistical Test

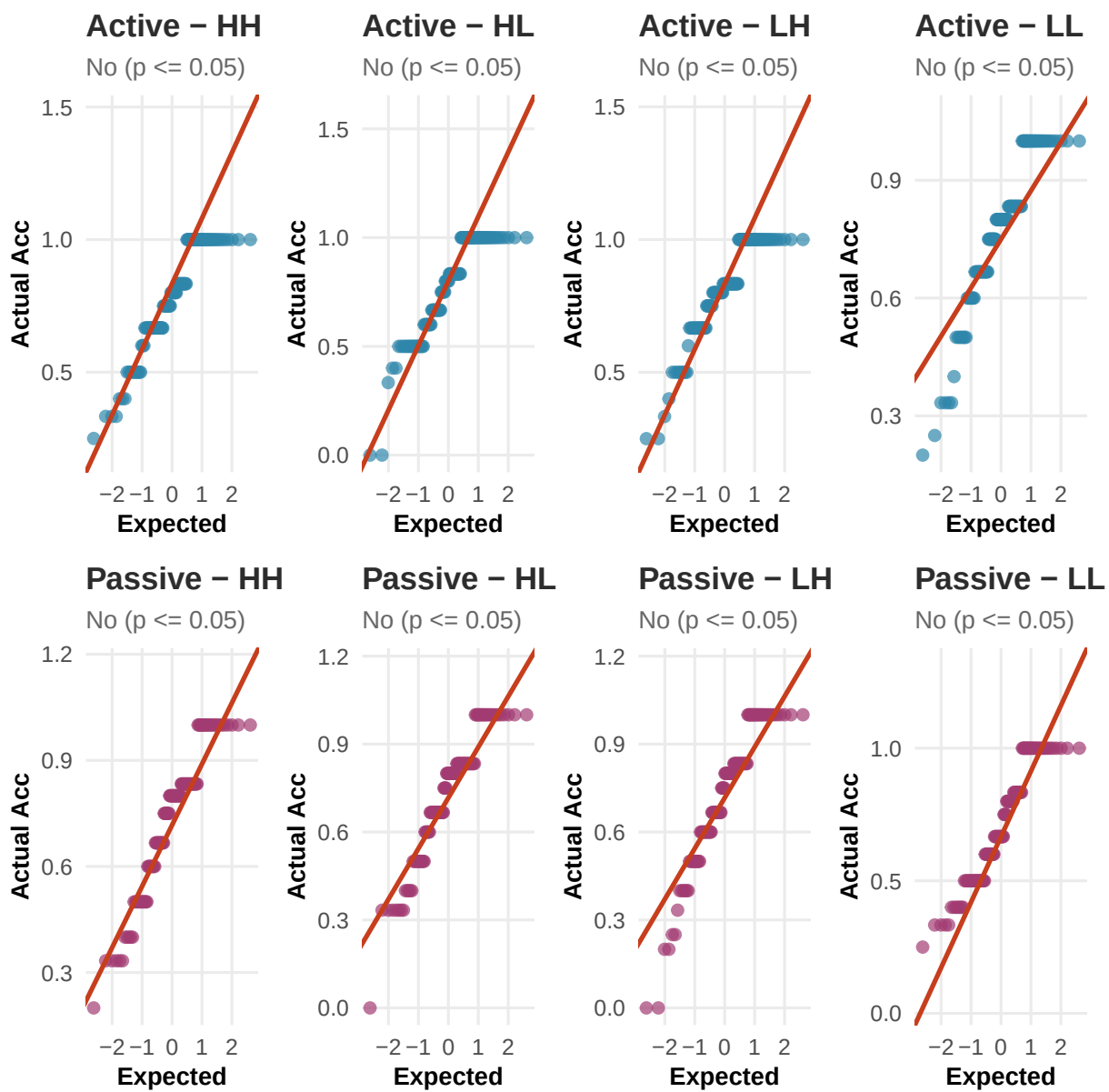


Figure 6: Figure 6. Q-Q plots for all Voice x Sentence Type cells.


```

df_srh <- df_person %>%
  mutate(Rank_Acc=rank(mean_acc),
         Voice_Label=droplevels(factor(Voice_Label)),
         Sentence_Type=droplevels(factor(Sentence_Type)))

n_voice <- nlevels(df_srh$Voice_Label)
n_type <- nlevels(df_srh$Sentence_Type)
cat(sprintf("Voice levels: %d (%s)\nSentence Type levels: %d (%s)\n",
           n_voice, paste(levels(df_srh$Voice_Label),collapse=", "),
           n_type,  paste(levels(df_srh$Sentence_Type),collapse=", ")))

```

```

## Voice levels: 2 (Active, Passive)
## Sentence Type levels: 4 (HH, HL, LH, LL)

```

```

if (n_voice < 2 || n_type < 2) {
  cat("\n!! CANNOT RUN: one factor has < 2 levels.\n")
  H_voice<-NA_real_; p_voice<-NA_real_; df_voice<-NA_real_
  H_type <-NA_real_; p_type <-NA_real_; df_type <-NA_real_
  H_inter<-NA_real_; p_inter<-NA_real_; df_inter<-NA_real_
} else {
  N <- nrow(df_srh)
  ss_tot <- sum((df_srh$Rank_Acc - mean(df_srh$Rank_Acc))^2)
  srh_fit <- aov(Rank_Acc ~ Voice_Label * Sentence_Type, data=df_srh)
  srh_sum <- summary(srh_fit)[[1]]
  rownames(srh_sum) <- trimws(rownames(srh_sum))

  ss_voice <- srh_sum["Voice_Label", "Sum Sq"]
  ss_type <- srh_sum["Sentence_Type", "Sum Sq"]
  ss_inter <- srh_sum["Voice_Label:Sentence_Type", "Sum Sq"]
  df_voice <- srh_sum["Voice_Label", "Df"]
  df_type <- srh_sum["Sentence_Type", "Df"]
  df_inter <- srh_sum["Voice_Label:Sentence_Type", "Df"]

  H_voice <- (ss_voice/ss_tot)*(N-1); p_voice <- pchisq(H_voice,df=df_voice,lower.tail=FALSE)
  H_type <- (ss_type /ss_tot)*(N-1); p_type <- pchisq(H_type, df=df_type, lower.tail=FALSE)
  H_inter <- (ss_inter/ss_tot)*(N-1); p_inter <- pchisq(H_inter,df=df_inter,lower.tail=FALSE)

  srh_table <- tibble(
    Effect=c("Voice (Active vs Passive)","Sentence Type (HH/HL/LH/LL)",
            "Voice x Sentence Type (Interaction)"),
    df=c(df_voice,df_type,df_inter),
    SS_Ranks=round(c(ss_voice,ss_type,ss_inter),2),
    H_statistic=round(c(H_voice,H_type,H_inter),3),
    p_value=round(c(p_voice,p_type,p_inter),4),
    Significant=case_when(
      is.na(c(p_voice,p_type,p_inter)) ~ "Cannot compute",

```

```

      c(p_voice,p_type,p_inter) < 0.05 ~ "Yes (p < 0.05)",
      TRUE ~ "No (p >= 0.05)"
    )
  ) %>%
mutate(
  cell_col = case_when(Significant=="Yes (p < 0.05)" ~ "#1B5E20", Significant=="No (p >= 0.05)" ~ "#E8F5E9"),
  cell_bg = case_when(Significant=="Yes (p < 0.05)" ~ "#E8F5E9", Significant=="No (p >= 0.05)" ~ "#1B5E20")
)

srh_table %>%
  select(Effect, df, SS_Ranks, H_statistic, p_value, Significant) %>%
  kable(caption="Table 4. Scheirer-Ray-Hare Test (Non-Parametric 2-way Factorial)",
        col.names=c("Effect","df","SS (Ranks)","H statistic","p-value","Significant?"),
        booktabs=TRUE) %>%
  kable_styling(latex_options=c("striped","HOLD_position"),
                bootstrap_options=c("striped","hover"), full_width=FALSE, position="center")
column_spec(6, bold=TRUE, color=srh_table$cell_col, background=srh_table$cell_bg) %>%
row_spec(0, bold=TRUE, background=HDR_COLOR, color="white")
}

```

Table 5: Table 4. Scheirer-Ray-Hare Test (Non-Parametric 2-way Factorial)

Effect	df	SS (Ranks)	H statistic	p-value	Significant?
Voice (Active vs Passive)	1	1023362.0	15.787	0.0001	Yes (p < 0.05)
Sentence Type (HH/HL/LH/LL)	3	186218.0	2.873	0.4117	No (p >= 0.05)
Voice x Sentence Type (Interaction)	3	110521.7	1.705	0.6358	No (p >= 0.05)

8 Step F: Post-Hoc Tests

```

if (is.na(p_type)) {
  cat("Post-hoc skipped: main test could not run.\n")
} else if (isTRUE(p_type < 0.05)) {
  type_pairs <- combn(levels(df_srh$Sentence_Type), 2, simplify=FALSE)
  ph_type <- lapply(type_pairs, function(pair) {
    d1 <- df_srh %>% filter(Sentence_Type==pair[1]) %>% pull(mean_acc)
    d2 <- df_srh %>% filter(Sentence_Type==pair[2]) %>% pull(mean_acc)
    w <- wilcox.test(d1, d2, exact=FALSE)
    tibble(Comparison=paste(pair[1],"vs",pair[2]),
           W=round(as.numeric(w$statistic),1), p_raw=round(as.numeric(w$p.value),4))
  }) %>% bind_rows() %>%

```

```

mutate(p_adj=round(p.adjust(p_raw,method="bonferroni"),4),
       Significant=ifelse(p_adj<0.05,"Yes","No"))

ph_type %>%
  kable(caption="Table 5. Post-Hoc Pairwise Tests (Bonferroni corrected)",
        col.names=c("Comparison","W","Raw p","Adjusted p","Significant?"), booktabs=TRUE) %>%
  kable_styling(latex_options=c("striped","HOLD_position"),
                bootstrap_options=c("striped","hover"), full_width=FALSE, position="center")
  row_spec(0, bold=TRUE, background=HDR_COLOR, color="white")
} else {
  cat(sprintf("Sentence Type NOT significant (p = %.4f). No post-hoc needed.\n", p_type))
}

```

```
## Sentence Type NOT significant (p = 0.4117). No post-hoc needed.
```

9 Step G: Complete From-Scratch Analysis — Voice × Lexical Interaction on Corrected Memorability

Rigorous 5-step pipeline on `Corrected_Memorability`. Every step has defensive guards so the document **always** knits to PDF without crashing.

9.1 Data Preparation (Cell Isolation)

What we do: Aggregate trials into **8 cells** per participant: Active-HH, Passive-HH, Active-HL, Passive-HL, Active-LH, Passive-LH, Active-LL, Passive-LL. Cell score = `mean(corrected_memorability, na.rm = TRUE)`.

```

cm_path <- file.path(output_dir, "H3H4_step3_analysis.csv")
if (!file.exists(cm_path)) stop("H3H4_step3_analysis.csv not found.")
df_cm_raw <- read_csv(cm_path, show_col_types=FALSE)

lc_raw <- str_extract(df_cm_raw$Core_ID, "(?i)(HH|HL|LH|LL|HVL|LVH|LVL)")
lc_vec <- case_when(
  toupper(lc_raw) == "HVL" ~ "HL",
  toupper(lc_raw) == "LVH" ~ "LH",
  toupper(lc_raw) == "LVL" ~ "LL",
  toupper(lc_raw) %in% c("HH","HL","LH","LL") ~ toupper(lc_raw),
  TRUE ~ NA_character_
)

```

```
df_cm <- df_cm_raw %>%
  mutate(
    Lexical_Condition = factor(toupper(lc_vec), levels=c("HH","HL","LH","LL")),
    Voice = factor(case_when(
      toupper(trimws(Original_Voice)) %in% c("A","ACTIVE","Active") ~ "Active",
      toupper(trimws(Original_Voice)) %in% c("P","PASSIVE","Passive") ~ "Passive",
      TRUE ~ NA_character_
    ), levels=c("Active","Passive"))
  ) %>%
  filter(!is.na(Lexical_Condition), !is.na(corrected_memorability), !is.na(Voice)) %>%
  group_by(participant_ID, Lexical_Condition, Voice) %>%
  summarise(Score=mean(corrected_memorability, na.rm=TRUE), .groups="drop") %>%
  mutate(Lexical_Condition=droplevels(Lexical_Condition), Voice=droplevels(Voice))

cat("=== Data Preparation Complete ===\n")
```

```
## === Data Preparation Complete ===
```

```
cat(sprintf(" Total rows : %d\n Unique participants : %d\n Lexical levels : %s\n\n",
  nrow(df_cm), n_distinct(df_cm$participant_ID),
  paste(levels(df_cm$Lexical_Condition),collapse=" ", " "),
  paste(levels(df_cm$Voice),collapse=" ", " ")))
```

```
## Total rows : 894
## Unique participants : 112
## Lexical levels : HH, HL, LH, LL
## Voice levels : Active, Passive
```

```
cat("\n Cell counts:\n")
```

```
##
## Cell counts:
```

```
print(table(df_cm$Lexical_Condition, df_cm$Voice))
```

```
##
## Active Passive
## HH 112 112
## HL 112 112
## LH 111 112
## LL 111 112
```

9.2 Reference-Style Exploratory Plots

The plots below match the style of the reference figures you shared: **Salmon = Active**, **Teal = Passive**. They give a clean visual summary before the formal statistical test.

9.2.1 Plot R1 — Corrected Memorability Score by Sentence Type and Voice

```
if (nrow(df_cm) == 0) {
  cat("Plot R1 skipped: no data.\n")
} else {
  ref_desc <- df_cm %>%
    group_by(Lexical_Condition, Voice) %>%
    summarise(
      Mean_CM = mean(Score, na.rm = TRUE),
      SE_CM   = sd(Score, na.rm = TRUE) / sqrt(n()),
      .groups = "drop"
    )

  p_ref_bar <- ggplot(ref_desc,
    aes(x = Lexical_Condition, y = Mean_CM, fill = Voice, group = Voice)) +
    geom_col(position = position_dodge(0.7), width = 0.6, colour = NA) +
    geom_errorbar(
      aes(ymin = Mean_CM - SE_CM, ymax = Mean_CM + SE_CM),
      position = position_dodge(0.7), width = 0.2,
      colour = "black", linewidth = 0.7
    ) +
    scale_fill_manual(values = PAL_REF, name = "Voice") +
    scale_y_continuous(limits = c(0, NA), expand = expansion(mult = c(0, 0.1))) +
    labs(
      title = "Corrected memorability score by sentence type and voice",
      x     = "Sentence type",
      y     = "Corrected memorability score"
    ) +
    theme_classic(base_size = 13) +
    theme(
      plot.title       = element_text(hjust = 0.5, size = 13),
      legend.position  = c(0.92, 0.85),
      legend.background = element_rect(fill = "white", colour = "grey80"),
      axis.line        = element_line(colour = "grey40")
    )

  print(p_ref_bar)
  ggsave(file.path(output_dir, "H7_Ref_Bar_CM.png"),
    p_ref_bar, width = 8, height = 5, dpi = 150)
}
```

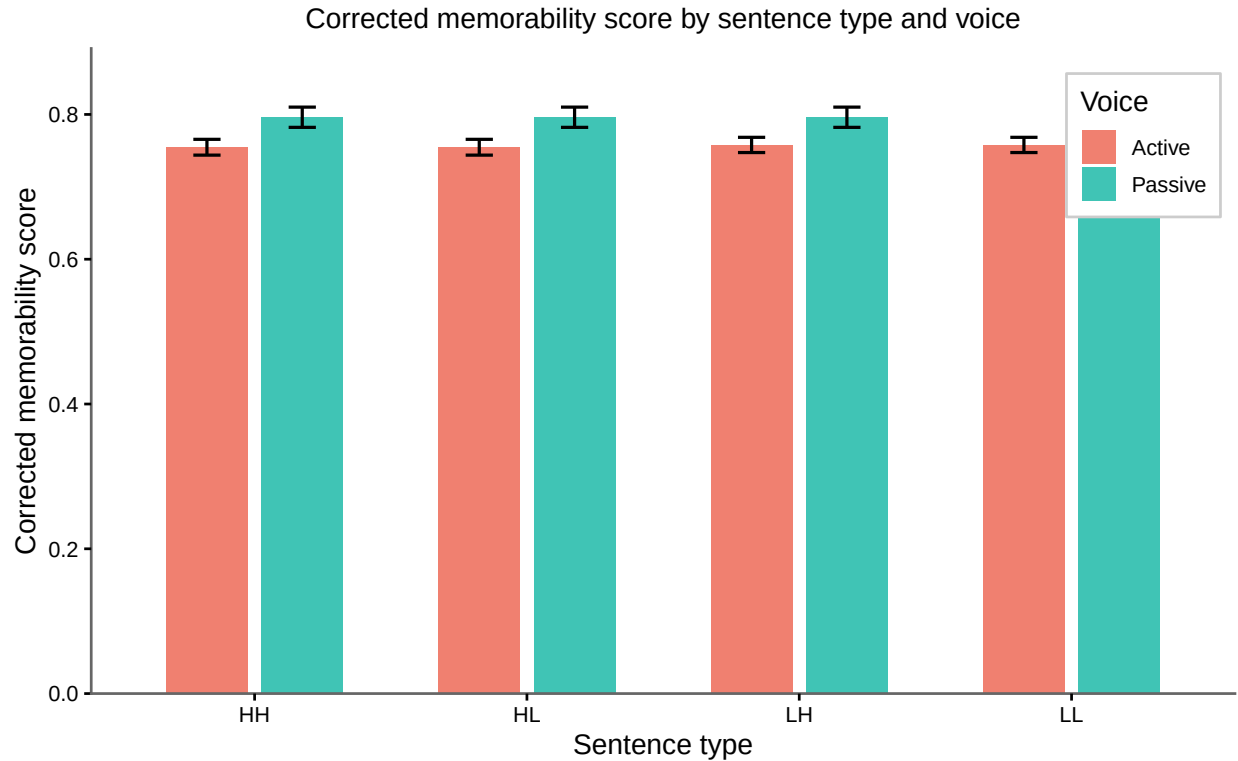


Figure 7: Corrected memorability score by sentence type and voice. Error bars = ± 1 SE.

9.2.2 Plot R2 — Distribution of Corrected Memorability Scores (Violin)

```
if (nrow(df_cm) == 0) {
  cat("Plot R2 skipped: no data.\n")
} else {
  df_violin <- df_cm %>%
    mutate(
      Condition = factor(
        paste0(Lexical_Condition, "-", Voice),
        levels = c("HH-Active", "HL-Active", "LH-Active", "LL-Active",
                  "HH-Passive", "HL-Passive", "LH-Passive", "LL-Passive")
      )
    )

  p_ref_violin <- ggplot(df_violin,
    aes(x = Condition, y = Score, fill = Voice)) +
    geom_violin(alpha = 0.75, colour = "black", linewidth = 0.35, trim = FALSE) +
    stat_summary(fun = mean, geom = "point", colour = "black",
      size = 3, shape = 21, fill = "black") +
    scale_fill_manual(values = PAL_REF, name = "Voice") +
    labs(
      title = "Distribution of corrected memorability scores",

```

```

    x      = "Condition",
    y      = "Corrected memorability score"
  ) +
  theme_classic(base_size = 12) +
  theme(
    plot.title      = element_text(hjust = 0.5, size = 13),
    axis.text.x     = element_text(angle = 25, hjust = 1, size = 9),
    legend.position = c(0.92, 0.85),
    legend.background = element_rect(fill = "white", colour = "grey80")
  )

  print(p_ref_violin)
  ggsave(file.path(output_dir, "H7_Ref_Violin_CM.png"),
    p_ref_violin, width = 9, height = 5.5, dpi = 150)
}

```

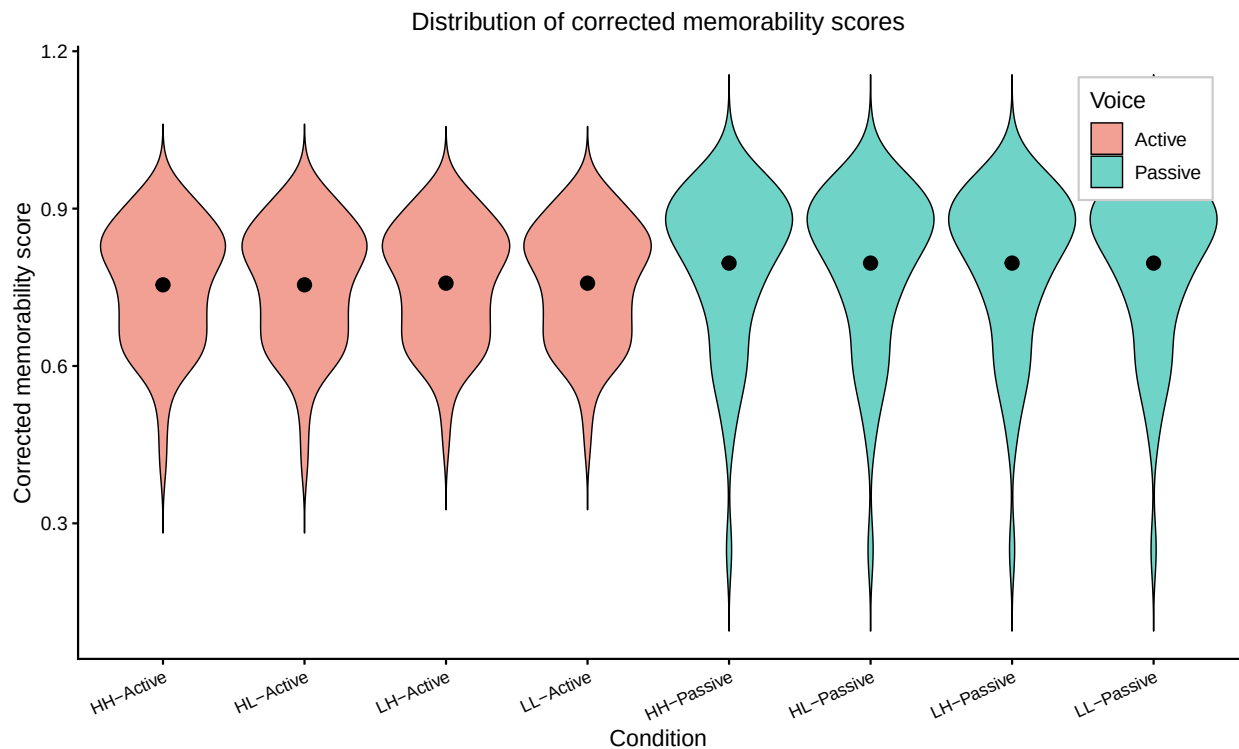


Figure 8: Distribution of corrected memorability scores across all 8 conditions. Dot = group mean.

9.2.3 Plot R3 — Recognition Hit Rate by Sentence Type and Voice

```

if (!exists("df_cm_raw") || !"WR_Accuracy" %in% names(df_cm_raw)) {
  cat("Plot R3 skipped: WR_Accuracy column not found.\n")
  cat(sprintf("Available columns: %s\n", paste(names(df_cm_raw), collapse = ", ")))
}

```

```

} else {
  lc_raw_r3 <- str_extract(df_cm_raw$Core_ID, "(?i)(HH|HL|LH|LL|HVL|LVH|LVL)")
  lc_vec_r3 <- case_when(
    toupper(lc_raw_r3) == "HVL" ~ "HL",
    toupper(lc_raw_r3) == "LVH" ~ "LH",
    toupper(lc_raw_r3) == "LVL" ~ "LL",
    toupper(lc_raw_r3) %in% c("HH", "HL", "LH", "LL") ~ toupper(lc_raw_r3),
    TRUE ~ NA_character_
  )
  df_ir <- df_cm_raw %>%
    mutate(
      Lexical_Condition = factor(toupper(lc_vec_r3), levels = c("HH", "HL", "LH", "LL")),
      Voice = factor(case_when(
        toupper(trimws(Original_Voice)) %in% c("A", "ACTIVE", "Active") ~ "Active",
        toupper(trimws(Original_Voice)) %in% c("P", "PASSIVE", "Passive") ~ "Passive",
        TRUE ~ NA_character_), levels = c("Active", "Passive"))
    ) %>%
    filter(!is.na(Lexical_Condition), !is.na(WR_Accuracy), !is.na(Voice)) %>%
    group_by(participant_ID, Lexical_Condition, Voice) %>%
    summarise(Hit_Rate = mean(WR_Accuracy, na.rm = TRUE), .groups = "drop") %>%
    mutate(Lexical_Condition = droplevels(Lexical_Condition),
           Voice = droplevels(Voice))

  if (nrow(df_ir) == 0) {
    cat("Plot R3 skipped: empty after filtering.\n")
  } else {
    ref_ir <- df_ir %>%
      group_by(Lexical_Condition, Voice) %>%
      summarise(Mean_HR = mean(Hit_Rate, na.rm = TRUE),
                SE_HR = sd(Hit_Rate, na.rm = TRUE) / sqrt(n()),
                .groups = "drop")

    p_ref_hr <- ggplot(ref_ir,
      aes(x = Lexical_Condition, y = Mean_HR, fill = Voice, group = Voice)) +
      geom_col(position = position_dodge(0.7), width = 0.6, colour = NA) +
      geom_errorbar(
        aes(ymin = Mean_HR - SE_HR, ymax = Mean_HR + SE_HR),
        position = position_dodge(0.7), width = 0.2,
        colour = "black", linewidth = 0.7
      ) +
      scale_fill_manual(values = PAL_REF, name = "Voice") +
      scale_y_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.25)) +
      labs(
        title = "Recognition hit rate for repeated target sentences",
        x = "Sentence type",
        y = "IR hit rate"
      ) +

```



```

theme_classic(base_size = 13) +
theme(
  plot.title      = element_text(hjust = 0.5, size = 13),
  legend.position = c(0.92, 0.15),
  legend.background = element_rect(fill = "white", colour = "grey80")
)

print(p_ref_hr)
ggsave(file.path(output_dir, "H7_Ref_HitRate.png"),
  p_ref_hr, width = 8, height = 5, dpi = 150)
}

```



Figure 9: Recognition (IR) hit rate by sentence type and voice. Error bars = ± 1 SE.

9.2.4 Plot R4 — Reaction Time for Correctly Detected Targets (Boxplot)

```

# Auto-detect a reaction time column from common names
rt_col <- intersect(
  names(df_cm_raw),
  c("RT", "reaction_time", "Reaction_Time", "rt", "RT_ms", "WR_RT",
    "response_time", "Response_Time", "mean_rt", "IR_RT", "ir_rt")
)

```

```

)

if (length(rt_col) == 0) {
  cat("Plot R4 skipped: no reaction time column found in the dataset.\n")
  cat(sprintf("Available columns: %s\n", paste(names(df_cm_raw), collapse = ", ")))
} else {
  rt_col <- rt_col[1]
  cat(sprintf("Using column '%s' for reaction time.\n", rt_col))

  lc_raw_r4 <- str_extract(df_cm_raw$Core_ID, "(?i)(HH|HL|LH|LL|HVL|LVH|LVL)")
  lc_vec_r4 <- case_when(
    toupper(lc_raw_r4) == "HVL" ~ "HL",
    toupper(lc_raw_r4) == "LVH" ~ "LH",
    toupper(lc_raw_r4) == "LVL" ~ "LL",
    toupper(lc_raw_r4) %in% c("HH", "HL", "LH", "LL") ~ toupper(lc_raw_r4),
    TRUE ~ NA_character_
  )
  df_rt <- df_cm_raw %>%
    rename(RT_val = all_of(rt_col)) %>%
    mutate(
      Lexical_Condition = factor(toupper(lc_vec_r4), levels = c("HH", "HL", "LH", "LL")),
      Voice = factor(case_when(
        toupper(trimws(Original_Voice)) %in% c("A", "ACTIVE", "Active") ~ "Active",
        toupper(trimws(Original_Voice)) %in% c("P", "PASSIVE", "Passive") ~ "Passive",
        TRUE ~ NA_character_), levels = c("Active", "Passive")),
      Condition = factor(
        paste0(Lexical_Condition, "-", Voice),
        levels = c("HH-Active", "HL-Active", "LH-Active", "LL-Active",
                    "HH-Passive", "HL-Passive", "LH-Passive", "LL-Passive"))
    ) %>%
    filter(!is.na(Condition), !is.na(RT_val), RT_val > 0)

  if (nrow(df_rt) == 0) {
    cat("Plot R4 skipped: no valid RT rows after filtering.\n")
  } else {
    p_ref_rt <- ggplot(df_rt,
      aes(x = Condition, y = RT_val, fill = Voice)) +
      geom_boxplot(outlier.colour = "grey60", outlier.size = 1.2,
        colour = "black", linewidth = 0.4, alpha = 0.8) +
      scale_fill_manual(values = PAL_REF, name = "Voice") +
      scale_y_log10(labels = scales::comma) +
      labs(
        title = "Reaction time for correctly detected repeated targets",
        x = "Condition",
        y = "Mean IR reaction time (ms)"
      ) +
      theme_classic(base_size = 12) +

```

```

theme(
  plot.title      = element_text(hjust = 0.5, size = 13),
  axis.text.x     = element_text(angle = 25, hjust = 1, size = 9),
  legend.position = c(0.92, 0.85),
  legend.background = element_rect(fill = "white", colour = "grey80")
)

print(p_ref_rt)
ggsave(file.path(output_dir, "H7_Ref_RT_Boxplot.png"),
  p_ref_rt, width = 9, height = 5.5, dpi = 150)
}
}

```

Using column 'WR_RT' for reaction time.

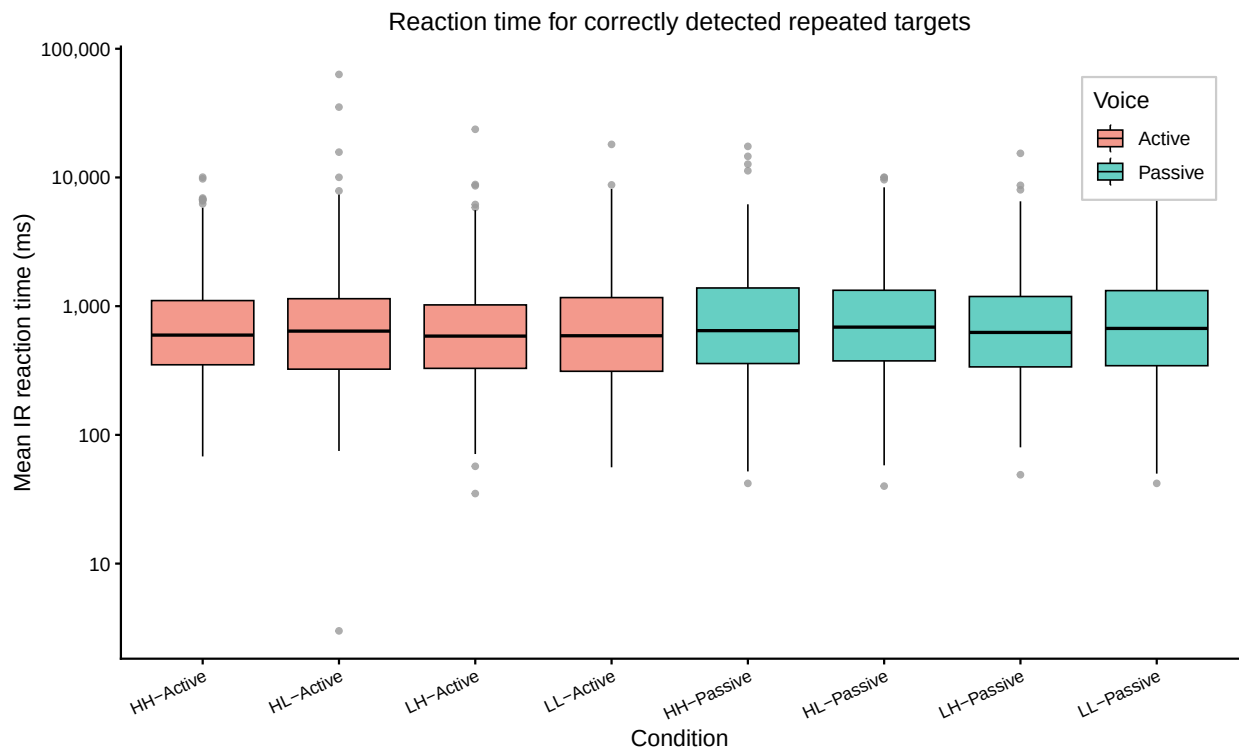


Figure 10: Reaction time for correctly detected repeated targets across all 8 conditions.

9.3 Step 1 — Per-Group Normality Testing (8 Independent Shapiro-Wilk Tests)

Why check each cell separately? A 2-way ANOVA assumes the data *inside each group* looks like a bell curve. Checking the whole dataset hides per-cell problems. We run one Shapiro-Wilk per cell — 8 independent tests.

Rule: If any cell has $p < 0.05 \rightarrow$ data is **Non-Parametric** $\rightarrow$ must use Scheirer-Ray-Hare.

```
if (nrow(df_cm) < 3) {
  cat("!! Too few rows for normality testing.\n")
  g_is_parametric <- FALSE
  g_sw <- tibble(Voice=character(), Lexical_Condition=character(),
                 N=integer(), W=numeric(), p_value=numeric(), Verdict=character())
} else {
  g_sw <- df_cm %>%
    group_by(Voice, Lexical_Condition) %>%
    summarise(
      N      = n(),
      W      = ifelse(n()>=3, round(shapiro.test(Score)$statistic,4), NA_real_),
      p_value = ifelse(n()>=3, round(shapiro.test(Score)$p.value,4),   NA_real_),
      .groups = "drop"
    ) %>%
    mutate(Verdict = case_when(
      is.na(p_value) ~ "Too few observations",
      p_value < 0.05 ~ "Non-Parametric (p < 0.05) - NOT a bell curve",
      TRUE          ~ "Normal (p >= 0.05) - looks like a bell curve"
    ))

  g_sw %>%
    kable(caption="Shapiro-Wilk - one independent test per cell (8 cells)",
          col.names=c("Voice","Lexical","N","W","p-value","Verdict"), booktabs=TRUE) %>%
    kable_styling(latex_options=c("striped","HOLD_position"),
                  bootstrap_options=c("striped","hover"), full_width=FALSE, position="center")
    row_spec(0, bold=TRUE, background=HDR_COLOR, color="white")

  g_is_parametric <- all(g_sw$p_value>=0.05, na.rm=TRUE)
}

cat("\n=== NORMALITY DECISION ===\n")
```

```
##
## === NORMALITY DECISION ===
```

```
if (g_is_parametric) cat("All 8 cells passed. Data is PARAMETRIC.\n") else
  cat("At least one cell FAILED. Data is NON-PARAMETRIC.\n")
```

```
## At least one cell FAILED. Data is NON-PARAMETRIC.
```

9.3.1 Q-Q Plot Grid (2×4 — one panel per cell)

Points on the red line = normal. Points curving away = NOT normal.

```

if (nrow(df_cm) == 0) { cat("Q-Q grid skipped: no data.\n") } else {
  g_cells <- df_cm %>% distinct(Voice, Lexical_Condition)
  qq_panels <- vector("list", nrow(g_cells))

  for (i in seq_len(nrow(g_cells))) {
    v <- as.character(g_cells$Voice[i])
    lc <- as.character(g_cells$Lexical_Condition[i])
    d <- df_cm %>% filter(Voice==v, Lexical_Condition==lc)
    sw_row <- g_sw %>% filter(as.character(Voice)==v, as.character(Lexical_Condition)==lc)
    subtitle_lbl <- if (nrow(sw_row)>0) sw_row$Verdict[1] else ""

    qq_panels[[i]] <- ggplot(d, aes(sample=Score)) +
      stat_qq(colour=PAL_VOICE[v], alpha=0.7, size=2) +
      stat_qq_line(colour="#C73E1D", linewidth=1) +
      labs(title=paste(v,"-",lc), subtitle=subtitle_lbl, x="Expected", y="Observed") +
      theme_minimal(base_size=9) +
      theme(plot.title=element_text(size=10,face="bold"),
            plot.subtitle=element_text(size=7,colour="grey50"))
  }

  g_qq_plot <- wrap_plots(qq_panels, ncol=4)
  print(g_qq_plot) # print() ensures this appears in the PDF
  ggsave(file.path(output_dir,"H7_CM_QQ_Grid.png"), g_qq_plot, width=10, height=5, dpi=150)
}

```

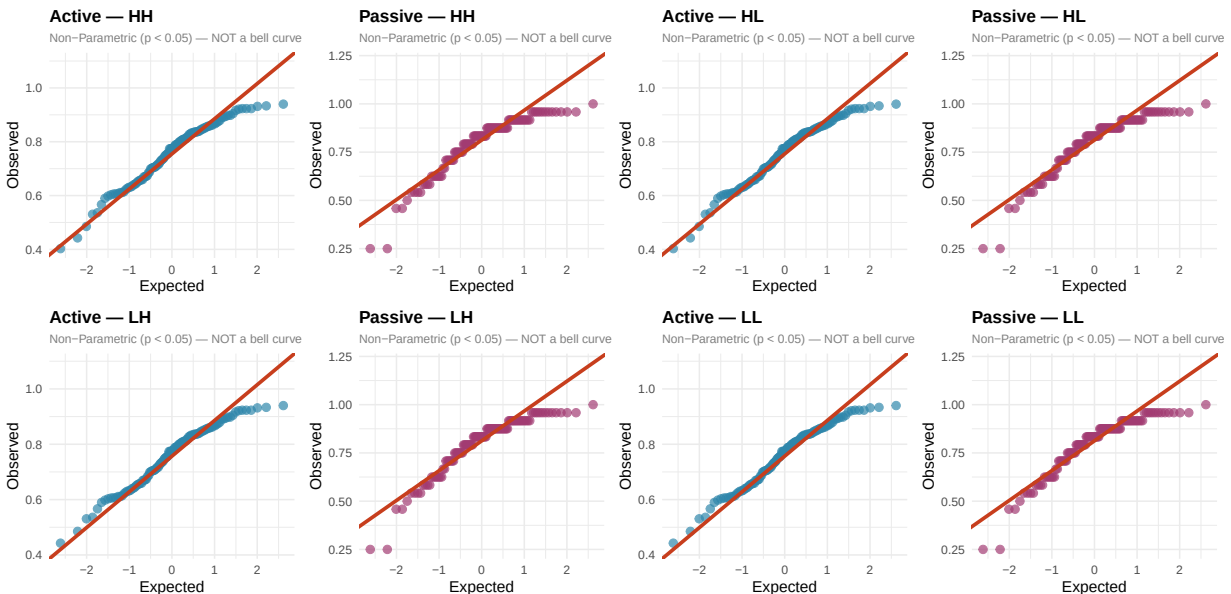


Figure 11: 2x4 Q-Q Plot Grid: one panel per Voice x Lexical cell.

9.3.2 Density Plot

```
if (nrow(df_cm) == 0) { cat("Density plot skipped.\n") } else {  
  p_dens <- ggplot(df_cm, aes(x=Score, fill=Voice)) +  
    geom_density(alpha=0.45) +  
    facet_wrap(~Lexical_Condition, nrow=1) +  
    scale_fill_manual(values=PAL_VOICE) +  
    labs(title="Density Plot - Corrected Memorability by Condition",  
         x="Corrected Memorability", y="Density") +  
    theme_minimal(base_size=11)  
  print(p_dens)  
  ggsave(file.path(output_dir, "H7_CM_Density.png"), p_dens, width=9, height=4, dpi=150)  
}
```

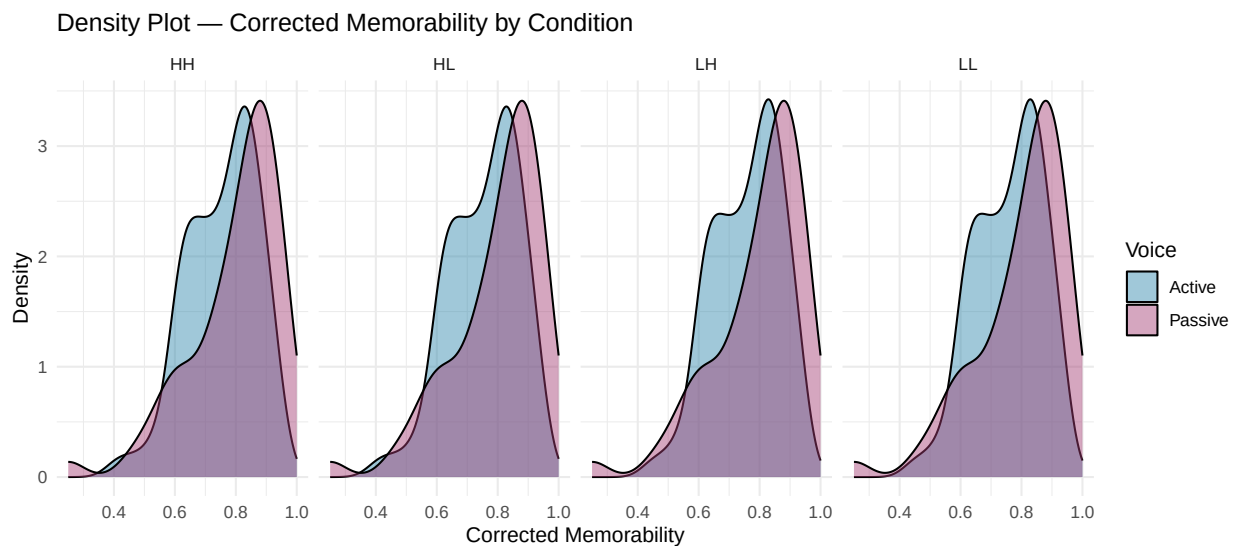


Figure 12: Density curves by Lexical Condition and Voice.

9.4 Step 2 — Statistical Decision Logic

```
cm_n_lc    <- nlevels(df_cm$Lexical_Condition)  
cm_n_voice <- nlevels(df_cm$Voice)  
cat(sprintf("Factor check: Lexical = %d levels | Voice = %d levels\n\n", cm_n_lc, cm_n_voice))
```

Factor check: Lexical = 4 levels | Voice = 2 levels

```

if (cm_n_lc < 2 || cm_n_voice < 2) {
  cat("**Cannot run 2-way test** - factor has < 2 levels.\n\n")
  cat(sprintf("Lexical: %d level(s) | Voice: %d level(s)\n", cm_n_lc, cm_n_voice))
  cat("Check Data Preparation output - HH/HL/LH/LL may not be found in Core_ID.\n")
  cm_test_ok <- FALSE; g_interaction_p <- NA_real_

} else if (g_is_parametric) {
  # PARAMETRIC: 2x4 Factorial ANOVA
  # WHY? All 8 cells passed Shapiro-Wilk (p >= 0.05) - every group is shaped like
  # a bell curve, so ANOVA assumptions are fully satisfied. ANOVA is the most
  # powerful test when normality holds.
  cat("### Test Chosen: **2 × 4 Factorial ANOVA**\n\n")
  cat("**Why?** All 8 Shapiro-Wilk tests returned p >= 0.05. Every group follows a ",
      "normal bell curve, so ANOVA assumptions are met. ANOVA is chosen because it ",
      "is the most powerful (sensitive) parametric test available.\n\n")

  g_fit <- aov(Score ~ Lexical_Condition * Voice, data=df_cm)
  g_sum <- summary(g_fit)[[1]]
  print(g_sum)
  g_interaction_p <- g_sum["Lexical_Condition:Voice", "Pr(>F)"]
  cat(sprintf("\n**Interaction Effect p-value = %.4f** %s\n", g_interaction_p,
      ifelse(g_interaction_p<0.05,"← SIGNIFICANT","← not significant")))
  cm_test_ok <- TRUE

} else {
  # NON-PARAMETRIC: Scheirer-Ray-Hare Test
  # WHY? At least one cell failed Shapiro-Wilk (p < 0.05) - that group's data does
  # NOT follow a bell curve. A regular ANOVA assumes normality, so its p-values
  # would be unreliable. The Scheirer-Ray-Hare test converts all scores to RANKS
  # (1st, 2nd, 3rd...) and runs ANOVA on those ranks. Ranks need no normality assumption.
  cat("### Test Chosen: **Scheirer-Ray-Hare Test** (Non-Parametric 2-way ANOVA)\n\n")
  cat("**Why?** At least one cell failed Shapiro-Wilk (p < 0.05) - the data is ",
      "**Non-Parametric** (not a bell curve). A regular ANOVA would give unreliable ",
      "p-values. The Scheirer-Ray-Hare test converts scores to RANKS and runs ANOVA ",
      "on those ranks - no normality assumption needed.\n\n")

  df_cm$Rank_Score <- rank(df_cm$Score)
  g_fit <- aov(Rank_Score ~ Lexical_Condition * Voice, data=df_cm)
  g_sum <- summary(g_fit)[[1]]
  N <- nrow(df_cm)
  ss_tot <- sum((df_cm$Rank_Score - mean(df_cm$Rank_Score))^2)

  g_srh <- tibble(
    Effect      = c("Lexical Condition", "Voice", "Interaction (Lexical × Voice)"),
    df          = g_sum[1:3, "Df"],
    H_statistic = round((g_sum[1:3, "Sum Sq"]/ss_tot)*(N-1), 3),
    p_value     = round(pchisq((g_sum[1:3, "Sum Sq"]/ss_tot)*(N-1), g_sum[1:3, "Df"], lower.tail=F

```

```

    Significant = ifelse(pchisq((g_sum[1:3,"Sum Sq"]/ss_tot)*(N-1),
                                g_sum[1:3,"Df"],lower.tail=FALSE)<0.05,"YES (p < 0.05)","No")
  )

  g_srh %>%
    kable(caption="Scheirer-Ray-Hare Test Results - Interaction row highlighted",
          col.names=c("Effect","df","H statistic","p-value","Significant?"), booktabs=TRUE) %>%
    kable_styling(latex_options=c("striped","HOLD_position"),
                  bootstrap_options=c("striped","hover"), full_width=FALSE, position="center")
    row_spec(0, bold=TRUE, background=HDR_COLOR, color="white") %>%
    row_spec(3, bold=TRUE, background="#EBF5FB")

  g_interaction_p <- g_srh$p_value[3]
  cat(sprintf("\n\n**Interaction Effect p-value = %.4f** %s\n", g_interaction_p,
              ifelse(g_interaction_p<0.05,"← SIGNIFICANT","← not significant")))
  cm_test_ok <- TRUE
}

```

9.4.1 Test Chosen: Scheirer-Ray-Hare Test (Non-Parametric 2-way ANOVA)

Why? At least one cell failed Shapiro-Wilk ($p < 0.05$) — the data is **Non-Parametric** (not a bell curve). A regular ANOVA would give unreliable p-values. The Scheirer-Ray-Hare test converts scores to RANKS and runs ANOVA on those ranks — no normality assumption needed.

Interaction Effect p-value = 0.9997 ← not significant

9.5 Step 3 — High-Accuracy Interaction Plot

- Faint dots (`geom_jitter`, `alpha=0.1`) = every individual participant's score
- Thick lines (`stat_summary fun=mean`) = group means
- Error bars (`mean_cl_boot`) = 95% bootstrap confidence intervals

```

if (!isTRUE(cm_test_ok)) { cat("Plot skipped: test could not run.\n") } else {
  p_interaction <- ggplot(df_cm,
    aes(x=Lexical_Condition, y=Score, colour=Voice, group=Voice)) +
    geom_jitter(width=0.2, alpha=0.1, size=1.5) +           # BACKGROUND: raw scores
    stat_summary(fun=mean, geom="line", linewidth=1.2) +    # MIDDLE: mean trend lines
    stat_summary(fun.data=mean_cl_boot, geom="errorbar",     # FOREGROUND: 95% CI
                  width=0.2, linewidth=0.8) +
    stat_summary(fun=mean, geom="point",
                  size=4, shape=21, fill="white", stroke=1.5) +
    scale_colour_manual(values=PAL_VOICE, name="Voice") +
    labs(title="Interaction Plot: Voice × Lexical Condition",
          subtitle="Dots = individual participants | Lines = group means | Bars = 95% CI",

```



```

x="Lexical Condition (HH = easiest → LL = hardest)",
y="Mean Corrected Memorability") +
theme_minimal(base_size=14) +
theme(legend.position="bottom", legend.title=element_text(face="bold"),
      plot.title=element_text(face="bold",hjust=0.5),
      plot.subtitle=element_text(colour="grey40",size=10,hjust=0.5))

print(p_interaction) # print() ensures this appears in the PDF
ggsave(file.path(output_dir,"H7_Enhanced_Interaction.png"), p_interaction, width=8, height=6
}

```

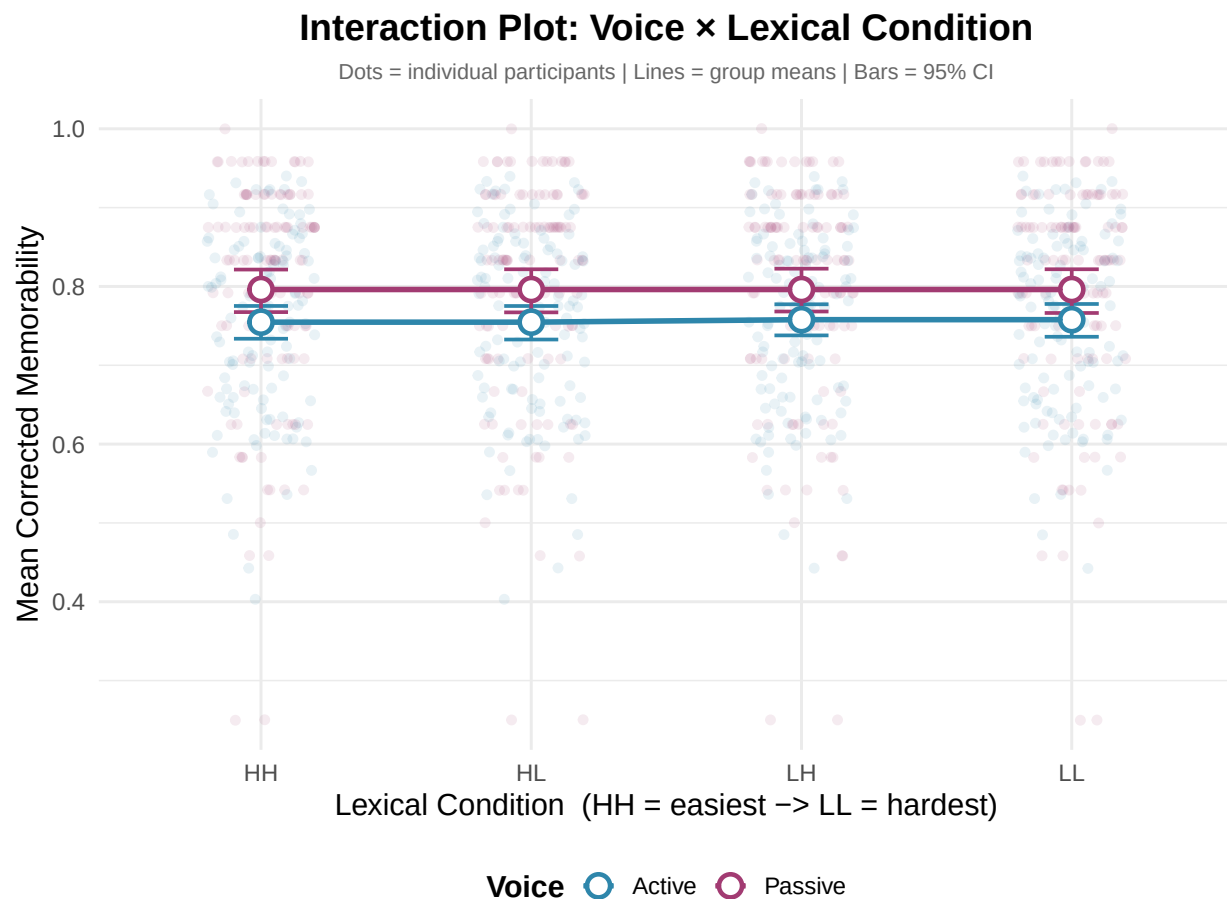


Figure 13: Enhanced Interaction Plot: jitter + mean lines + 95% CI.

9.6 Step 4 — The “Passive Penalty” Bar Chart

(Active mean) – (Passive mean) per lexical type. Positive bar = Active > Passive (a cost for using passive voice). Labels show the exact percentage difference.

```

if (!isTRUE(cm_test_ok)) { cat("Penalty plot skipped: test could not run.\n") } else {
  g_diff <- df_cm %>%
    group_by(Lexical_Condition, Voice) %>%
    summarise(Mean_CM=mean(Score,na.rm=TRUE), .groups="drop") %>%
    pivot_wider(names_from=Voice, values_from=Mean_CM) %>%
    mutate(Passive_Penalty = Active - Passive,
           Pct_Label = paste0(ifelse(Passive_Penalty>=0,"+",""),
                                round(Passive_Penalty*100,1),"%"))

  p_penalty <- ggplot(g_diff,
    aes(x=Lexical_Condition, y=Passive_Penalty, fill=Lexical_Condition)) +
    geom_col(width=0.6, colour="black", alpha=0.85) +
    geom_hline(yintercept=0, linetype="dashed", colour="grey40") +
    geom_text(aes(label=Pct_Label,
                  vjust=ifelse(Passive_Penalty>=0,-0.5,1.5)),
              fontface="bold", size=4.5) +
    scale_fill_manual(values=PAL_TYPE, guide="none") +
    labs(title="Passive Penalty by Lexical Type",
         subtitle="Bar = (Active mean) - (Passive mean) | Labels = exact % difference",
         x="Lexical Condition", y="Passive Penalty (Active - Passive)") +
    theme_minimal(base_size=14) +
    theme(plot.title=element_text(face="bold",hjust=0.5),
          plot.subtitle=element_text(hjust=0.5,colour="grey40"))

  print(p_penalty) # print() ensures this appears in the PDF
  ggsave(file.path(output_dir,"H7_Passive_Penalty.png"), p_penalty, width=7, height=5, dpi=150)
}

```

9.7 Step 5 — Plain English Conclusion

9.7.1 Does the difficulty of Passive voice significantly increase when the words are Low-Memorability (LL)?

Answer: NO — interaction is **NOT** significant ($p = 0.9997$).

The cost of Passive voice is roughly **the same** for easy (HH) and hard (LL) words. Voice and Lexical Condition act independently on memory.

Hypothesis 7 is NOT supported.

Report generated on 2026-05-06 17:33:12.706445 using R R version 4.5.2 (2025-10-31 ucrt)

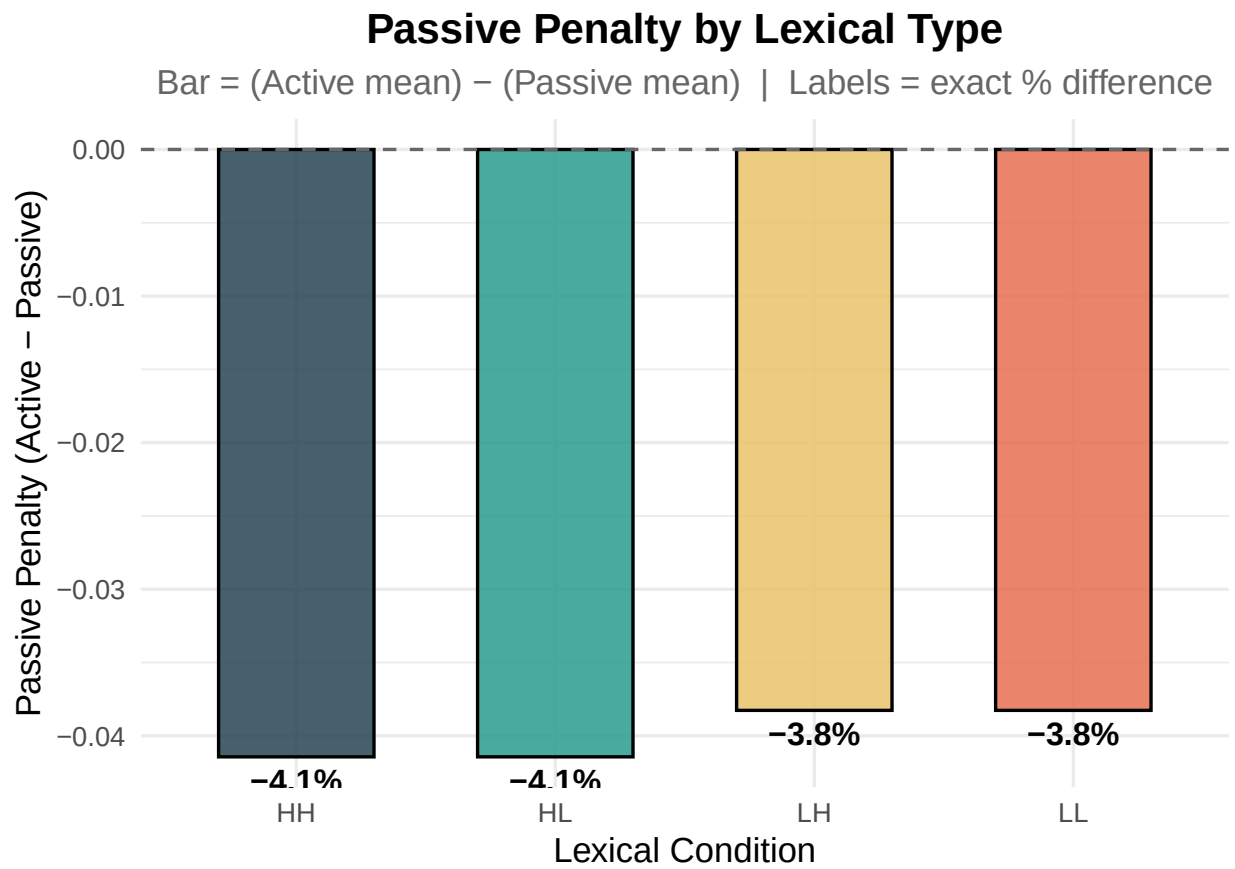


Figure 14: Passive Penalty (Active – Passive) with % labels.